Euristiche MIP

Indice

- Euristiche basate sulla PLI
- Gli esempi del capitolo
- Arrotondamento
- Diving
- La feasibility pump
- Migliorare l'incumbent con un sotto-MIP
  - RINS
  - Local branching
  - Proximity search
- Relax-and-fix e fix-and-optimize
  - Relax-and-fix
  - Fix-and-optimize
- Le euristiche dentro il solver
- Che cosa danno le euristiche basate sulla PLI
- Implementazione in `Python`
- Errori tipici
- In sintesi

Obiettivi

Alla fine di questo capitolo sarai in grado di:

- spiegare come un algoritmo euristico usa il rilassamento lineare e un solver MIP;
- eseguire a mano l'arrotondamento, il diving, la feasibility pump, RINS, il local branching e la proximity search su un piccolo MIP;
- eseguire relax-and-fix e fix-and-optimize su un MIP le cui variabili sono suddivise in blocchi;
- decidere quale metodo usare quando manca una soluzione ammissibile, o quando una soluzione incumbent va migliorata;
- calcolare il gap di una soluzione euristica quando è disponibile un bound duale.

Prerequisiti

- Da questo modulo, i capitoli «Bound primali e duali» e «Algoritmi branch-and-bound basati su LP»: il rilassamento lineare, i lower bound e gli upper bound, il gap di ottimalità e le operazioni di branching
- Da questo modulo, il capitolo «Solver MIP: che cosa fanno»: la soluzione incumbent, i sotto-problemi risolti da un solver e le euristiche al suo interno
- Dal modulo *Programmazione lineare*: l'algoritmo del simplesso duale (Capitolo 10), che riottimizza un LP dopo la modifica di un limite su una variabile

Euristiche basate sulla PLI

Introduzione: Euristiche basate sulla PLI

- Il branch-and-bound trova buone soluzioni solo per caso, come foglie dell'albero: spesso una buona soluzione serve **presto**, e in fretta
- Un'euristica basata sulla PLI usa due strumenti di questo modulo: il **rilassamento lineare**, per decidere dove cercare, e il **solver MIP**, per esplorare in modo esatto un insieme ampio di soluzioni
- Fissiamo la notazione del capitolo: il MIP in forma generale, l'incumbent, l'arrotondamento e il completamento di un punto
- Incontriamo le cinque idee alla base di tutti i metodi del capitolo

Definizione 1: MIP in forma generale (1/2)

*Un **MIP** con N variabili $\boldsymbol{x} = (x_1, x_2, \ldots, x_N)$ si scrive*

$$\begin{aligned} \min \quad & f(\boldsymbol{x}) \\ \text{soggetto a} \quad & A\boldsymbol{x} \leq b, \\ & l_j \leq x_j \leq u_j, \quad \forall j \in \{1, 2, \ldots, N\}, \\ & x_j \in \mathbb{Z}, \qquad\qquad \forall j \in \mathscr{D}, \end{aligned}$$

*dove f è una funzione lineare e $\mathscr{D} \subseteq \{1, 2, \ldots, N\}$ è l'insieme degli indici delle **variabili intere**.*

Definizione: MIP in forma generale (2/2)

La regione ammissibile del ***rilassamento lineare*** *è*

$$\mathcal{L} = \left\{ \boldsymbol{x} \in \mathbb{R}^N : A\,\boldsymbol{x} \leq b,\ l \leq \boldsymbol{x} \leq u \right\},$$

e

$$z(\mathrm{LP}) = \min\{ f(\boldsymbol{x}) : \boldsymbol{x} \in \mathcal{L} \}$$

è il valore del rilassamento lineare.

- Un problema di massimizzazione, come l'esempio dell'auto, si tratta nello stesso modo, scambiando min e max, $\leq$ e $\geq$ nei confronti fra valori della funzione obiettivo

Definizione 2: euristica basata sulla PLI

*Un **algoritmo euristico** restituisce una soluzione ammissibile di un problema MIP senza alcuna garanzia di ottimalità. È **basato sulla PLI** (una **matheuristica**) se usa il rilassamento lineare e un solver MIP per cercare la soluzione.*

- Un'euristica fornisce un bound primale (un upper bound per un problema di minimizzazione, un lower bound per un problema di massimizzazione), mai un bound duale
- L'algoritmo branch-and-bound di questo modulo trova la soluzione incumbent solo per caso, come foglia dell'albero

I due strumenti di un'euristica basata sulla PLI

- La soluzione ottima del **rilassamento lineare** indica *dove cercare*: è la soluzione di un problema più semplice, spesso vicina alle buone soluzioni del problema originale

- Il **solver MIP** esplora *in modo esatto* un insieme di soluzioni troppo grande per essere enumerato, purché l'insieme sia descritto da vincoli lineari

Definizione 3: incumbent, arrotondamento e completamento

- *Una soluzione ammissibile $\bar{\mathbf{x}}$ del MIP, la migliore nota, è la soluzione **incumbent**, e $f(\bar{\mathbf{x}})$ è il suo valore.*
- *L'**arrotondamento** di un numero reale v all'intero più vicino è $[\,v\,] = \lfloor v + \frac{1}{2} \rfloor$, così che un valore $\frac{1}{2}$ viene arrotondato per eccesso.*
- *Un punto $\bar{\mathbf{x}}$ con $\bar{x}_j \in \mathbb{Z}$ per ogni $j \in \mathscr{D}$ viene **completato** fissando le variabili intere ai valori $\bar{x}_j$ e risolvendo l'LP sulle variabili continue.*

Notazione del capitolo

- In tutto il capitolo, $\tilde{\boldsymbol{x}}$ indica una soluzione ottima di un rilassamento lineare, e $\bar{\boldsymbol{x}}$ un punto le cui componenti $\bar{x}_j$, $j \in \mathscr{D}$, sono intere
- Se l'LP del completamento ha una soluzione ottima, $\bar{\boldsymbol{x}}$ diventa una soluzione ammissibile del MIP; se è inammissibile, lo è anche $\bar{\boldsymbol{x}}$
- Quando tutte le variabili sono intere, il completamento si limita a verificare i vincoli

Cinque idee

- I metodi si basano su cinque idee, una o due per sezione:

idea	metodi
arrotondare il rilassamento	arrotondamento; diving (fissare una variabile, risolvere di nuovo, ripetere)
inseguire l'ammissibilità	la feasibility pump
fissare dove rilassamento e incumbent concordano	RINS
limitare la distanza dall'incumbent	local branching; proximity search
scomporre in blocchi di variabili	relax-and-fix; fix-and-optimize

Il solver degli esempi

- Gli esempi sono risolti con il solver MIP Gurobi, usato da Python tramite `gurobipy`
- Il modulo *Modellazione MIP* (fabiofurini.github.io/modellazione-mip, capitolo 3 «Dal modello a Python/Gurobi») spiega come usarlo
- Quando un LP ha più soluzioni ottime, riportiamo quella restituita dal solver
- Tutti i codici sono nella sezione sull'implementazione in Python

Riepilogo

- Un'**euristica basata sulla PLI** (matheuristica) trova una soluzione ammissibile con il rilassamento lineare e un solver MIP, senza alcuna garanzia di ottimalità: fornisce solo un bound primale

- $\tilde{\boldsymbol{x}}$ è una soluzione LP ottima, $\bar{\boldsymbol{x}}$ un punto intero su $\mathscr{D}$; l'**incumbent** è la migliore soluzione ammissibile nota

- $[\,v\,] = \lfloor v + \frac{1}{2} \rfloor$ è l'arrotondamento; il **completamento** fissa le variabili intere e risolve l'LP sulle variabili continue

- Cinque idee: arrotondare, inseguire l'ammissibilità, fissare dove LP e incumbent concordano, limitare la distanza, scomporre in blocchi

Gli esempi del capitolo

Introduzione: Gli esempi del capitolo

- Tutti i metodi del capitolo vengono eseguiti sugli stessi piccoli problemi, così che si possano confrontare
- Ritroviamo l'esempio dell'auto, e quattro piccoli problemi del catalogo dei problemi MIP
- Per ogni problema conosciamo il valore LP, l'upper bound e il valore ottimo: possiamo sempre misurare quanto è buona una soluzione euristica

L'esempio dell'auto e il catalogo

- I metodi sono illustrati sull'**esempio dell'auto** di questo modulo e su quattro piccoli problemi del **catalogo dei problemi MIP**
- Sono tutti problemi PLI in forma canonica di massimizzazione, con dati interi e $x_j \in \mathbb{Z}_{\geq 0}$ per ogni j, così che $\mathscr{D} = \{1, 2, \ldots, N\}$
- L'**esempio dell'auto** si scrive:

$$\begin{array}{rrcrcr} \max & 3\,x_1 & + & 2\,x_2 & & \\ \text{soggetto a} & 10\,x_1 & + & 4\,x_2 & \leq & 45 \\ & 4\,x_1 & + & 14\,x_2 & \leq & 49 \\ & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0} \end{array}$$

Il problema *P7*

$$
\begin{aligned}
\max \quad & 3x_1 + x_2 \\
\text{soggetto a} \quad & x_1 + 4x_2 \leq 12, \\
& 2x_1 + x_2 \leq 5, \\
& x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2\}
\end{aligned}
$$

Il problema $P42$

$$
\begin{aligned}
\max \quad & 3x_1 + 2x_2 + 4x_3 \\
\text{soggetto a} \quad & x_1 + x_2 + 2x_3 \leq 4, \\
& 2x_1 + 3x_3 \leq 5, \\
& 2x_1 + x_2 + 3x_3 \leq 7, \\
& x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, 3\}
\end{aligned}
$$

Il problema $P46$

$$
\begin{aligned}
\max \quad & 5x_1 + 6x_2 + 10x_3 \\
\text{soggetto a} \quad & 9x_1 + 2x_2 + 2x_3 \leq 11, \\
& 2x_1 + 6x_2 + 6x_3 \leq 16, \\
& 4x_1 + 2x_2 + 7x_3 \leq 10, \\
& x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, 3\}
\end{aligned}
$$

Il problema *P72*

$$
\begin{aligned}
\max \quad & 7x_1 + 6x_2 + 5x_3 - 2x_4 + 3x_5 \\
\text{soggetto a} \quad & x_1 + 3x_2 + 5x_3 - 2x_4 + 2x_5 \leq 4, \\
& 4x_1 + 2x_2 - 2x_3 + x_4 + x_5 \leq 3, \\
& 2x_1 + 4x_2 + 4x_3 - 2x_4 + 5x_5 \leq 5, \\
& 3x_1 + x_2 + 2x_3 - x_4 - 2x_5 \leq 1, \\
& x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \ldots, 5\}
\end{aligned}
$$

Bound e valori ottimi

- La tabella riporta il valore $z(\mathrm{LP})$ del rilassamento lineare, l'upper bound $\lfloor z(\mathrm{LP}) \rfloor$ (la funzione obiettivo assume valori interi) e il valore ottimo $z(\mathrm{ILP})$
- Tutti calcolati con l'algoritmo del simplesso in aritmetica esatta e verificati con Gurobi

problema	variabili	vincoli	$z(\mathrm{LP})$	$\lfloor z(\mathrm{LP}) \rfloor$	$z(\mathrm{ILP})$
auto	2	2	$\frac{31}{2}$	15	14
*P*7	2	2	$\frac{15}{2}$	7	7
*P*42	3	3	$\frac{21}{2}$	10	10
*P*46	3	3	$\frac{499}{25}$	19	16
*P*72	5	4	$\frac{507}{59}$	8	7

Esempio: le soluzioni LP degli esempi

problema	soluzione ottima $\tilde{\boldsymbol{x}}$ del rilassamento lineare
auto	$\left(\frac{7}{2}, \frac{5}{2}\right)$
$P7$	$\left(\frac{5}{2}, 0\right)$
$P42$	$\left(\frac{5}{2}, \frac{3}{2}, 0\right)$
$P46$	$\left(\frac{17}{25}, \frac{49}{25}, \frac{12}{25}\right)$
$P72$	$\left(\frac{39}{59}, 0, \frac{91}{59}, \frac{166}{59}, \frac{37}{59}\right)$

- Le prime tre soluzioni hanno al più due componenti frazionarie, con parte frazionaria $\frac{1}{2}$
- Le ultime due sono gli esempi in cui i metodi hanno più occasioni di fallire

Riepilogo

- Cinque piccoli problemi PLI in forma canonica di massimizzazione: l'**esempio dell'auto**, *P*7, *P*42, *P*46 e *P*72, tutti con $\mathscr{D} = \{1, 2, \ldots, N\}$
- Per ognuno conosciamo $z(\mathrm{LP})$, l'upper bound $\lfloor z(\mathrm{LP}) \rfloor$ e $z(\mathrm{ILP})$
- Il bound è stretto su *P*7 e *P*42, non sull'esempio dell'auto, su *P*46 e su *P*72
- *P*46 e *P*72 hanno soluzioni LP con molte componenti frazionarie: lì i metodi hanno più occasioni di fallire

Arrotondamento

Introduzione: Arrotondamento

- L'idea più semplice: risolvere il rilassamento lineare e arrotondarne la soluzione
- Vediamo quando l'arrotondamento funziona, quando fallisce, e un caso in cui l'arrotondamento *per difetto* è sempre sicuro
- Sugli esempi, l'arrotondamento all'intero più vicino fallisce tre volte: è la motivazione dei metodi delle prossime sezioni

Definizione 4: arrotondamento

L'arrotondamento (rounding) è la più semplice euristica basata sulla PLI. Risolve il rilassamento lineare, arrotonda ogni variabile intera della soluzione LP all'intero più vicino e completa le variabili continue. Restituisce una soluzione ammissibile, oppure fallisce se il punto arrotondato viola un vincolo.

- Lo pseudocodice dell'arrotondamento è nelle prossime due slide

Algorithm: Arrotondamento (1/2)

Input : MIP con variabili intere $\mathscr{D}$
Output : Soluzione ammissibile $\bar{\boldsymbol{x}}$, oppure FALLIMENTO

1 **function** *Rounding(MIP)*
2 $\quad \tilde{\boldsymbol{x}} :=$ soluzione ottima del rilassamento lineare
$\quad\quad$ // $\min\{f(\boldsymbol{x}) : \boldsymbol{x} \in \mathcal{L}\}$
3 $\quad$ **for** $j \in \mathscr{D}$ **do**
4 $\quad\quad \bar{x}_j \leftarrow [\tilde{x}_j]$ // arrotonda all'intero più vicino
5 $\quad$ **end**
6 $\quad$ fissa $x_j := \bar{x}_j$ per ogni $j \in \mathscr{D}$ e risolvi l'LP sulle variabili
$\quad\quad$ continue

Algorithm: Arrotondamento (2/2)

function *Rounding(MIP)* (continua)

```
7     if l'LP è ammissibile then
8         return x̄ completato con la parte continua della
            soluzione LP
9     else
10        return FALLIMENTO   // il punto arrotondato
            viola un vincolo
11    end
12 end
```

Perché l'arrotondamento può funzionare

- Quando la formulazione è **forte**, la soluzione LP è vicina a una soluzione intera ottima
- Molte delle sue variabili intere sono già intere
- L'arrotondamento ne cambia solo poche, ciascuna di meno di $\frac{1}{2}$

Perché l'arrotondamento può fallire

- L'arrotondamento tratta le variabili una alla volta, mentre i vincoli le legano fra loro
- Un vincolo con tutti i coefficienti non negativi e termine noto $b_i \geq 0$ resta soddisfatto se le variabili vengono arrotondate *per difetto*, e uno della forma $\geq$ se vengono arrotondate *per eccesso*
- Un vincolo di **uguaglianza** è quasi sempre violato da arrotondamenti indipendenti: due valori frazionari con somma 1 diventano, per esempio, 1 e 1

- Quando tutti i dati sono non negativi, l'arrotondamento per difetto non può violare un vincolo:

Osservazione 1

Siano tutti i coefficienti di A non negativi, $b \geq 0$ e $l = 0$. Se $\tilde{\boldsymbol{x}} \in \mathcal{L}$, allora il punto $\lfloor \tilde{\boldsymbol{x}} \rfloor$, ottenuto arrotondando per difetto ogni variabile intera, appartiene a $\mathcal{L}$.

Dimostrazione

- **Limiti.** Per ogni j:

$$0 \leq \lfloor \tilde{x}_j \rfloor \leq \tilde{x}_j \leq u_j,$$

quindi i limiti sono rispettati

- **Vincoli.** Per ogni riga i, poiché $a_{ij} \geq 0$:

$$\sum_j a_{ij} \lfloor \tilde{x}_j \rfloor \leq \sum_j a_{ij} \tilde{x}_j \leq b_i$$

□

L'esempio dell'auto soddisfa le ipotesi dell'osservazione:
l'arrotondamento per difetto è sempre ammissibile
e, in generale, non ottimo.

Vediamo che cosa succede sugli esempi.

Esempio (1/3): arrotondamento sull'esempio dell'auto

- La soluzione LP $\tilde{\boldsymbol{x}} = \left(\frac{7}{2}, \frac{5}{2}\right)$ viene arrotondata a $\bar{\boldsymbol{x}} = (4, 3)$, che viola il primo vincolo:

$$10 \cdot 4 + 4 \cdot 3 = 52 > 45$$

- **L'arrotondamento fallisce**

- L'arrotondamento per difetto dà invece $(3, 2)$:

$$10 \cdot 3 + 4 \cdot 2 = 38 \leq 45, \qquad 4 \cdot 3 + 14 \cdot 2 = 40 \leq 49,$$

  di valore 13, mentre $z(\mathrm{ILP}) = 14$

Esempio (2/3): arrotondamento su $P42$

- La soluzione LP $\left(\frac{5}{2}, \frac{3}{2}, 0\right)$ viene arrotondata a $(3, 2, 0)$, che viola il secondo vincolo:

$$2 \cdot 3 = 6 > 5$$

- **L'arrotondamento fallisce**

- L'arrotondamento per difetto dà $(2, 1, 0)$, di valore 8, mentre $z(\mathrm{ILP}) = 10$

Esempio (3/3): arrotondamento su $P72$

- La soluzione LP $\left(\frac{39}{59}, 0, \frac{91}{59}, \frac{166}{59}, \frac{37}{59}\right)$ viene arrotondata a $(1, 0, 2, 3, 1)$, che viola il primo vincolo:

$$1 + 10 - 6 + 2 = 7 > 4$$

- **L'arrotondamento fallisce**

Riepilogo

- L'**arrotondamento** risolve il rilassamento lineare, arrotonda le variabili intere all'intero più vicino e completa quelle continue
- Fallisce quando più variabili sono frazionarie contemporaneamente e i vincoli sono stringenti: esempio dell'auto, $P42$, $P72$
- Con dati non negativi, l'**arrotondamento per difetto** è sempre ammissibile, ma lascia per strada una parte del valore dell'LP: 13 contro 14 sull'esempio dell'auto, 8 contro 10 su $P42$

Diving

Introduzione: Diving

- L'arrotondamento prende tutte le sue decisioni in una volta, sulla base di un'unica soluzione LP
- Il **diving** le prende una alla volta, e dopo ogni decisione interroga di nuovo l'LP: la soluzione LP viene ricalcolata con le decisioni prese fino a quel momento, e si adatta a esse
- Vediamo due regole (diving frazionario e diving guidato), il legame con il branch-and-bound, e diving che riescono e diving che falliscono

Definizione 5: diving

Il diving risolve ripetutamente il rilassamento lineare, sceglie una variabile intera con valore frazionario, la fissa a un valore intero e risolve di nuovo l'LP. Si ferma con una soluzione ammissibile quando la soluzione LP è intera su tutte le variabili intere, e fallisce quando l'LP diventa inammissibile. Una regola decide quale variabile fissare e a quale valore.

- Lo pseudocodice del diving è nelle prossime due slide

Algorithm: Diving (1/2)

Input : MIP; una regola; per il diving guidato, una soluzione incumbent $\bar{\boldsymbol{x}}$

Output : Soluzione ammissibile, oppure FALLIMENTO

```
1 function Diving(MIP, regola)
2     while vero do
3         risolvi il rilassamento lineare con i limiti correnti l, u
4         if l'LP è inammissibile then
5             return FALLIMENTO   // il diving finisce
                in un LP inammissibile
6         end
7         x̃ := soluzione ottima dell'LP
8         F := {j ∈ 𝒟 : x̃_j ∉ ℤ}        // variabili intere
            frazionarie
```

Algorithm: Diving (2/2)

function *Diving(MIP, regola)* (continua)

    while *vero* (continua) **do**
        if $F = \emptyset$ **then**
            return $\tilde{x}$ `// intera su` $\mathscr{D}$`: ammissibile`
        end
        scegli $j \in F$ e un valore $v \in \mathbb{Z}$ secondo la regola
        $l_j \leftarrow v$; $u_j \leftarrow v$ `// fissa` $x_j := v$
    end
end

Due regole

- **Diving frazionario.** Si sceglie la variabile *meno* frazionaria, cioè la $j \in F$ che minimizza $\left|\tilde{x}_j - [\,\tilde{x}_j\,]\right|$, e la si fissa a $v = [\,\tilde{x}_j\,]$
  - L'LP è quasi sicuro del valore di questa variabile, quindi fissarla cambia la soluzione LP il meno possibile
  - Nei codici di questo capitolo, i pareggi si risolvono a favore della variabile di indice più piccolo

- **Diving guidato** (Danna, Rothberg e Le Pape, 2005). Data una soluzione incumbent $\bar{\boldsymbol{x}}$, si sceglie la $j \in F$ che minimizza $|\tilde{x}_j - \bar{x}_j|$ e la si fissa a $v = \bar{x}_j$
  - L'incumbent guida il diving verso una regione di buone soluzioni, e l'LP decide in quale ordine adottarne i valori

Altre regole, e il legame con il branch-and-bound

- Esistono molte altre regole: il *coefficient diving* fissa la variabile che compare nel minor numero di vincoli che il suo arrotondamento può violare; il *pseudo-cost diving* usa la storia dell'albero di branch-and-bound; il *vector-length diving* è pensato per i vincoli di set covering. I solver moderni ne eseguono diversi

- **Diving e branch-and-bound.** Fissare $x_j := [\,\tilde{x}_j\,]$ è una **decisione di branching**

- Un diving è un unico cammino dalla radice dell'albero di branch-and-bound a una foglia, che prende sempre uno dei due figli, senza mai tornare indietro: una **ricerca in profondità senza backtracking**

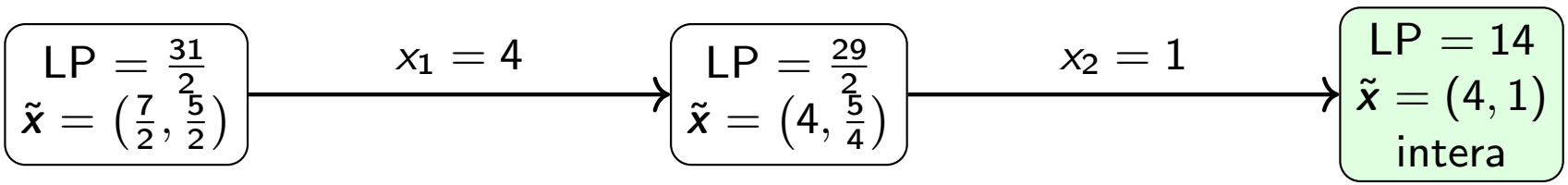
Costo e fallimento

- **Costo.** Ogni passo risolve un LP che differisce dal precedente per un solo limite su una variabile, quindi l'algoritmo del simplesso duale lo riottimizza partendo dalla base precedente in poche iterazioni
- Un diving con $|\mathscr{D}|$ variabili intere richiede al più $|\mathscr{D}|$ LP, di solito molti meno, perché una soluzione LP ha già molte componenti intere
- **Fallimento.** L'LP diventa inammissibile quando le fissazioni fatte fino a quel momento sono incompatibili
- Le implementazioni allora annullano l'ultima fissazione e provano l'altro valore (un solo backtracking), oppure rinunciano

Esempio (1/2): diving frazionario sull'esempio dell'auto

- **Passo 1.** La soluzione LP è $\tilde{\boldsymbol{x}} = \left(\frac{7}{2}, \frac{5}{2}\right)$, di valore $\frac{31}{2}$. Entrambe le variabili sono a distanza $\frac{1}{2}$ dall'intero più vicino; il pareggio si risolve con l'indice: $x_1 := \left[\frac{7}{2}\right] = 4$
- **Passo 2.** Con $x_1 = 4$ i vincoli diventano $4x_2 \leq 5$ e $14x_2 \leq 33$, quindi la soluzione LP è $\tilde{\boldsymbol{x}} = \left(4, \frac{5}{4}\right)$, di valore $12 + \frac{5}{2} = \frac{29}{2}$. Si fissa $x_2 := \left[\frac{5}{4}\right] = 1$
- **Passo 3.** La soluzione LP $(4, 1)$, di valore 14, è intera: il diving restituisce $(4, 1)$, che è la soluzione **ottima** del problema PLI

Esempio (2/2): diving frazionario sull'esempio dell'auto



- Il diving è un unico cammino dell'albero di branch-and-bound: due fissazioni, tre LP

Esempio (1/2): diving guidato sull'esempio dell'auto

- La soluzione incumbent è $\bar{\boldsymbol{x}} = (3, 2)$, di valore 13 (arrotondamento per difetto)
- Nella soluzione LP $\left(\frac{7}{2}, \frac{5}{2}\right)$ entrambe le variabili sono a distanza $\frac{1}{2}$ da $\bar{\boldsymbol{x}}$; si fissa $x_1 := 3$
- La nuova soluzione LP è $\left(3, \frac{37}{14}\right)$, di valore $\frac{100}{7}$; si fissa $x_2 := \bar{x}_2 = 2$
- La soluzione LP $(3, 2)$ è intera, di valore 13: il diving **restituisce l'incumbent**, senza migliorarlo

Esempio (2/2): diving guidato su $P7$

- La soluzione incumbent è $\bar{\boldsymbol{x}} = (2, 0)$, di valore 6 (arrotondamento per difetto di $\left(\frac{5}{2}, 0\right)$)
- L'unica variabile frazionaria è x_1: si fissa $x_1 := 2$
- La soluzione LP $(2, 1)$, di valore 7, è intera: il diving guidato trova la soluzione **ottima** dopo una sola fissazione
- Questo avviene anche se l'incumbent ha $\bar{x}_2 = 0$, perché l'LP ricostruisce le variabili che non sono fissate

L'incumbent decide solo i *valori*
delle variabili fissate:

sull'esempio dell'auto entrambi i valori fissati
coincidono con quelli dell'incumbent scadente,
su $P7$ il valore fissato è quello giusto.

Esempio (1/2): un diving che fallisce su $P42$

- La soluzione LP è $\left(\frac{5}{2}, \frac{3}{2}, 0\right)$; x_1 e x_2 sono a distanza $\frac{1}{2}$, e il pareggio si risolve con l'indice: $x_1 := \left[\frac{5}{2}\right] = 3$
- Il secondo vincolo dà

$$2 \cdot 3 + 3x_3 \leq 5,$$

  violato per ogni $x_3 \geq 0$
- L'LP è inammissibile: **il diving fallisce al primo passo**

Esempio (2/2): un diving che fallisce su $P72$

- Le componenti frazionarie della soluzione LP sono $\frac{39}{59}, \frac{91}{59}, \frac{166}{59}, \frac{37}{59}$ (per x_1, x_3, x_4, x_5)
- La meno frazionaria è x_4, a distanza $\frac{11}{59}$ da 3: $x_4 := 3$
- Il nuovo LP ha valore $\frac{728}{85}$ e soluzione $\left(\frac{56}{85}, 0, \frac{138}{85}, 3, \frac{52}{85}\right)$
- La variabile meno frazionaria è x_1, arrotondata a 1
- Fissare $x_1 := 1$ e $x_4 := 3$ rende l'LP **inammissibile**

- In entrambi i casi le prime fissazioni seguono l'LP, che non vede che i vincoli, tutti insieme, non lasciano spazio
- Un diving che fallisce non dà nessuna soluzione
- È questa la ragione della feasibility pump della prossima sezione

Riepilogo

- Il **diving** fissa una variabile frazionaria alla volta e risolve di nuovo l'LP: un unico cammino dell'albero di branch-and-bound, senza backtracking
- Il **diving frazionario** fissa la variabile meno frazionaria al suo arrotondamento; il **diving guidato** fissa la variabile più vicina all'incumbent al valore dell'incumbent
- Ogni passo costa poco: il simplesso duale riottimizza partendo dalla base precedente
- Sull'esempio dell'auto il diving frazionario trova l'ottimo $(4, 1)$; su $P42$ e $P72$ il diving finisce in un LP inammissibile

La feasibility pump

Introduzione: La feasibility pump

- L'arrotondamento e il diving falliscono su $P42$ e su $P72$, e su molti MIP falliscono la maggior parte delle volte: per un MIP generale, anche solo *decidere se esiste una soluzione ammissibile* è NP-completo
- La **feasibility pump** di Fischetti, Glover e Lodi (2005) rinuncia alla funzione obiettivo e si concentra su un unico scopo: trovare una soluzione **ammissibile**
- Definiamo la distanza da un punto intero, la pompa con le sue perturbazioni, e la eseguiamo su tre esempi

Due punti che si inseguono

- L'idea è lavorare con due punti contemporaneamente:
  - un punto di $\mathcal{L}$ soddisfa tutti i vincoli lineari, ma può essere frazionario
  - un punto intero è intero, ma può violare i vincoli
- Una soluzione ammissibile è un punto che ha entrambe le proprietà
- La pompa fa inseguire i due punti finché non coincidono, e per farlo deve misurare quanto sono lontani l'uno dall'altro

Definizione 6: Distanza da un punto intero

Sia $\bar{\boldsymbol{x}}$ *un punto con* $\bar{x}_j \in \mathbb{Z}$ *per ogni* $j \in \mathscr{D}$. *La* ***distanza*** *di un punto* $\boldsymbol{x} \in \mathbb{R}^N$ *da* $\bar{\boldsymbol{x}}$ *è*

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = \sum_{j \in \mathscr{D}} |x_j - \bar{x}_j|,$$

la distanza L_1 *calcolata solo sulle variabili intere.*

La distanza per variabili binarie

- Quando tutte le variabili intere sono **binarie**, $|x_j - \bar{x}_j|$ vale x_j se $\bar{x}_j = 0$ e $1 - x_j$ se $\bar{x}_j = 1$, per ogni $x_j \in [0, 1]$
- La distanza è allora una funzione *lineare* di $\boldsymbol{x}$:

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = \sum_{j \in \mathscr{D}:\ \bar{x}_j = 0} x_j + \sum_{j \in \mathscr{D}:\ \bar{x}_j = 1} (1 - x_j)$$

per variabili binarie

La distanza per variabili intere generali

- Per una variabile intera generale vale lo stesso se $\bar{x}_j = l_j$ (il termine è $x_j - l_j$) o $\bar{x}_j = u_j$ (il termine è $u_j - x_j$)
- Altrimenti $|x_j - \bar{x}_j|$ si sostituisce con una variabile ausiliaria d_j con i due vincoli

$$d_j \geq x_j - \bar{x}_j, \qquad d_j \geq \bar{x}_j - x_j;$$

  all'ottimo, $d_j = |x_j - \bar{x}_j|$
- In tutti i casi, il punto di $\mathcal{L}$ più vicino a $\bar{\boldsymbol{x}}$ si calcola risolvendo **un LP**

Definizione 7: feasibility pump (1/2)

La feasibility pump costruisce due successioni di punti:

- *punti* $\tilde{\boldsymbol{x}}^0, \tilde{\boldsymbol{x}}^1, \tilde{\boldsymbol{x}}^2, \ldots$ *di* $\mathcal{L}$*, che soddisfano i vincoli lineari;*
- *punti interi* $\bar{\boldsymbol{x}}^0, \bar{\boldsymbol{x}}^1, \bar{\boldsymbol{x}}^2, \ldots,$ *che soddisfano i vincoli di integralità.*

Parte dalla soluzione LP $\tilde{\boldsymbol{x}}^0$*.*

Definizione: feasibility pump (2/2)

Ogni punto intero è l'arrotondamento dell'ultimo punto di $\mathcal{L}$, e ogni punto di $\mathcal{L}$ è la proiezione dell'ultimo punto intero, cioè il punto di $\mathcal{L}$ a distanza minima da esso. Si ferma quando la distanza è 0: i due punti coincidono e sono una soluzione ammissibile. Quando l'arrotondamento restituisce un punto intero già visitato, la pompa lo perturba.

- Lo pseudocodice della feasibility pump è nelle prossime tre slide

Algorithm: Feasibility pump (1/3)

Input : MIP; numero massimo di iterazioni *maxIter*; seme dei numeri casuali

Output : Soluzione ammissibile, oppure FALLIMENTO

```
1  function FeasibilityPump(MIP, maxIter, seed)
2      x̃ := soluzione ottima del rilassamento lineare
3      if x̃_j ∈ ℤ per ogni j ∈ 𝒟 then
4          return x̃
5      end
6      x̄ ← [x̃]          // arrotonda le componenti intere
7      for it := 1 to maxIter do
8          x̃ ← arg min{Δ(x, x̄) : x ∈ ℒ}  // proiezione: un
             LP
9          if Δ(x̃, x̄) = 0 then
10             return x̃                  // x̃ è intero su 𝒟:
                 ammissibile
11         end
12         x̂ ← [x̃]
```

Algorithm: Feasibility pump (2/3)

function *FeasibilityPump(MIP, maxIter, seed)* (continua)

    for $it := 1$ **to** *maxIter* (continua) **do**
13   **if** $\hat{x}_j = \bar{x}_j$ *per ogni* $j \in \mathscr{D}$ **then** // ciclo corto
14          $j \leftarrow \arg\max_{k \in \mathscr{D}} |\tilde{x}_k - \bar{x}_k|$
15   sposta $\hat{x}_j$ di un'unità verso $\tilde{x}_j$ // flip
16   **end**
17   **if** $\hat{\boldsymbol{x}}$ *coincide con uno degli ultimi* 3 *punti* $\bar{\boldsymbol{x}}$ **then**
          // ciclo lungo
18   **for** $j \in \mathscr{D}$ **do**
19              $\rho_j \leftarrow$ numero casuale uniforme in $[-0.3, 0.7]$
20   **if** $|\tilde{x}_j - \hat{x}_j| + \max\{\rho_j, 0\} > \frac{1}{2}$ **then**
21                  $\hat{x}_j \leftarrow 1 - \hat{x}_j$ // restart
22   **end**
23   **end**
24   **end**

Algorithm: Feasibility pump (3/3)

function *FeasibilityPump(MIP, maxIter, seed)* (continua)
    for $it := 1$ **to** *maxIter* (continua) **do**
25      $\bar{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}}$
26 **end**
27 **return** *FALLIMENTO*
28 **end**

Perché la pompa funziona

- Ogni **proiezione** avvicina il punto di $\mathcal{L}$ all'ultimo punto intero
- Ogni **arrotondamento** porta il punto intero su quello più vicino
- La distanza fra le due successioni tende a diminuire, e i punti vengono *pompati* verso una regione in cui $\mathcal{L}$ contiene punti interi
- Ogni iterazione costa un LP e un arrotondamento

Cicli corti

- Se l'arrotondamento della proiezione restituisce lo stesso punto intero, la pompa ripeterebbe all'infinito gli stessi due punti
- Cambia allora le componenti di $\bar{\boldsymbol{x}}$ su cui $\tilde{\boldsymbol{x}}$ è più incerto, quelle con il $|\tilde{x}_j - \bar{x}_j|$ più grande
- La pompa originale fa il flip di un numero casuale di componenti compreso fra 10 e 30
- Sulle piccole istanze di queste slide basta un flip: il codice cambia la sola componente con il $|\tilde{x}_j - \bar{x}_j|$ più grande (la prima, in caso di pareggio)

Cicli lunghi e numeri casuali

- **Cicli lunghi.** La pompa può anche ciclare fra più punti interi. Quando il nuovo punto intero coincide con uno degli ultimi, la pompa originale esegue un **restart**:
  - una componente binaria viene invertita quando $|\tilde{x}_j - \hat{x}_j| + \max\{\rho_j, 0\} > \frac{1}{2}$, con ρ_j casuale uniforme in $[-0.3, 0.7]$
  - le componenti in cui $\tilde{x}_j$ è lontano da $\hat{x}_j$ vengono invertite con alta probabilità

- **Numeri casuali.** Nei codici di questo capitolo, sono prodotti dal generatore *minimal standard* di Park e Miller, con aritmetica intera a 64 bit:

$$seed \leftarrow (16807 \cdot seed) \bmod (2^{31} - 1), \qquad u = \frac{seed}{2^{31} - 1} \in (0, 1),$$

e nella pompa $\rho_j = u - 0.3$. Fissare il seme iniziale rende ogni esecuzione riproducibile

La funzione obiettivo e le variabili intere generali

- **La funzione obiettivo.** Dopo il primo LP, la funzione obiettivo originale f viene ignorata: la pompa trova una soluzione ammissibile, ma non necessariamente buona
- La *objective feasibility pump* di Achterberg e Berthold (2007) minimizza una combinazione

$$(1-\alpha)\,\Delta(\boldsymbol{x},\bar{\boldsymbol{x}})+\alpha\, f(\boldsymbol{x})$$

  con un peso α che diminuisce a ogni iterazione: le soluzioni trovate sono molto migliori, al prezzo di qualche iterazione in più

- **Variabili intere generali.** La pompa è nata per variabili binarie, dove il flip cambia $\bar{x}_j$ in $1-\bar{x}_j$; per variabili intere generali sposta $\bar{x}_j$ di un'unità verso $\tilde{x}_j$, e la distanza usa le variabili ausiliarie d_j (Bertacco, Fischetti e Lodi, 2007)

Eseguiamo ora la pompa a mano:
prima su un problema in due variabili, dove possiamo disegnare
le due successioni di punti,
poi sull'esempio dell'auto e su $P72$.

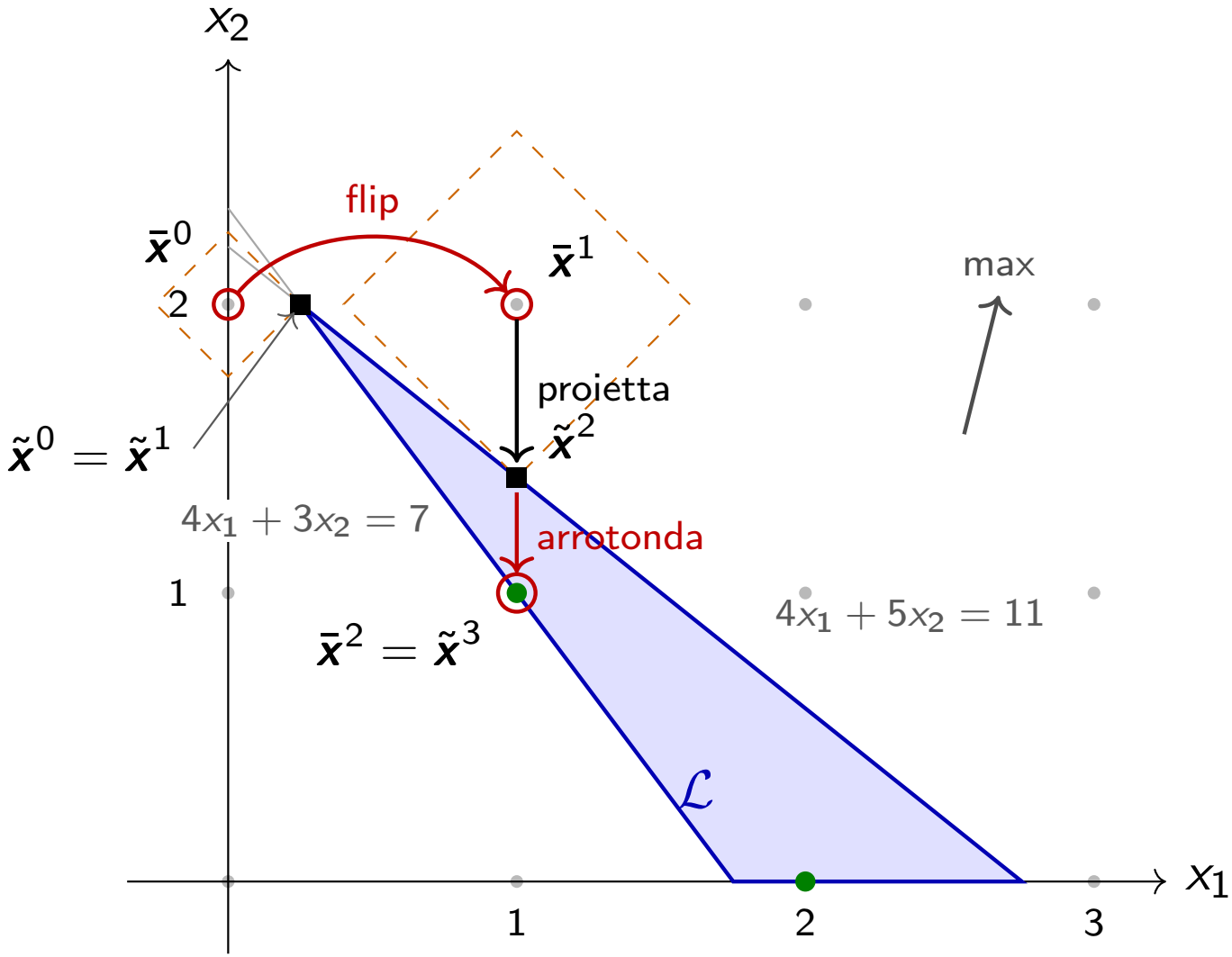
Esempio (1/7): la feasibility pump in due variabili

- Consideriamo il programma intero

$$\begin{aligned} \max \quad & x_1 + 4\,x_2 \\ \text{soggetto a} \quad & 4\,x_1 + 3\,x_2 \geq 7 \\ & 4\,x_1 + 5\,x_2 \leq 11 \\ & x_1 \in \{0, 1, \ldots, 5\} \\ & x_2 \in \{0, 1, \ldots, 5\} \end{aligned}$$

- La regione $\mathcal{L}$ è il triangolo di vertici $\left(\frac{1}{4}, 2\right)$, $\left(\frac{7}{4}, 0\right)$, $\left(\frac{11}{4}, 0\right)$, che contiene solo due punti interi, $(1, 1)$ e $(2, 0)$

Esempio (2/7): la feasibility pump in due variabili



Quadrati: i punti di $\mathcal{L}$; cerchi: i punti interi; tratteggiati: i rombi $\{x : \Delta(x, \bar{x}) \leq r\}$ delle due proiezioni.

Esempio (3/7): la feasibility pump in due variabili

- Le iterazioni sono le seguenti:

k	punto intero	proiezione $\tilde{\boldsymbol{x}}^k$	$\Delta(\tilde{\boldsymbol{x}}^k, \bar{\boldsymbol{x}}^{k-1})$	punto intero successivo
0	—	ottimo LP $(\frac{1}{4}, 2)$	—	arrotondamento: $\bar{\boldsymbol{x}}^0 = (0, 2)$
1	$\bar{\boldsymbol{x}}^0 = (0, 2)$	$(\frac{1}{4}, 2)$	$\frac{1}{4}$	$[\tilde{\boldsymbol{x}}^1] = \bar{\boldsymbol{x}}^0$: flip di x_1, $\bar{\boldsymbol{x}}^1 = (1, 2)$
2	$\bar{\boldsymbol{x}}^1 = (1, 2)$	$(1, \frac{7}{5})$	$\frac{3}{5}$	arrotondamento: $\bar{\boldsymbol{x}}^2 = (1, 1)$
3	$\bar{\boldsymbol{x}}^2 = (1, 1)$	$(1, 1)$	0	stop: ammissibile

Esempio (4/7): la feasibility pump in due variabili

- **Iterazione 0.** L'ottimo LP è il vertice $\left(\frac{1}{4}, 2\right)$, di valore $\frac{33}{4}$. Il suo arrotondamento $(0, 2)$ viola il primo vincolo:

$$4 \cdot 0 + 3 \cdot 2 = 6 < 7$$

- **Iterazione 1.** Con $\bar{\boldsymbol{x}}^0 = (0, 2)$, x_1 è al suo limite inferiore e

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^0) = x_1 + |x_2 - 2|$$

Esempio (5/7): la feasibility pump in due variabili

- **Iterazione 1 (continua).** La distanza L_1 di un punto da un segmento che non è né orizzontale, né verticale, né di pendenza ± 1 è minima in un estremo o dove il segmento incontra la retta orizzontale o quella verticale passante per il punto
- I candidati sono quindi i tre vertici e queste intersezioni
- La retta orizzontale $x_2 = 2$ tocca $\mathcal{L}$ solo nel vertice $\left(\frac{1}{4}, 2\right)$, e la retta verticale $x_1 = 0$ non lo incontra
- Il punto più vicino è $\tilde{\boldsymbol{x}}^1 = \left(\frac{1}{4}, 2\right)$, a distanza $\frac{1}{4}$

Esempio (6/7): la feasibility pump in due variabili

- **Iterazione 1 (continua).** L'arrotondamento di $\tilde{\boldsymbol{x}}^1$ è di nuovo $(0, 2) = \bar{\boldsymbol{x}}^0$: un **ciclo corto**
- La componente con il $|\tilde{x}_j^1 - \bar{x}_j^0|$ più grande è x_1 ($\frac{1}{4}$ contro 0), che si sposta di un'unità verso $\frac{1}{4}$: $\bar{\boldsymbol{x}}^1 = (1, 2)$
- **Iterazione 2.** Il punto $(1, 2)$ viola il secondo vincolo ($14 > 11$)
- La retta verticale $x_1 = 1$ attraversa $\mathcal{L}$ fra $(1, 1)$ e $\left(1, \frac{7}{5}\right)$, a distanze 1 e $\frac{3}{5}$ da $(1, 2)$; la retta orizzontale $x_2 = 2$ dà $\left(\frac{1}{4}, 2\right)$, a distanza $\frac{3}{4}$; gli altri due vertici sono più lontani
- Quindi $\tilde{\boldsymbol{x}}^2 = \left(1, \frac{7}{5}\right)$, e il suo arrotondamento $(1, 1)$ è un punto nuovo

Esempio (7/7): la feasibility pump in due variabili

- **Iterazione 3.** Il punto $(1, 1)$ appartiene a $\mathcal{L}$:

$$4 + 3 = 7 \geq 7, \qquad 4 + 5 = 9 \leq 11$$

- La sua distanza da $\mathcal{L}$ è 0, e la pompa restituisce $(1, 1)$, di valore 5

- È **ottima**, perché l'altro punto intero $(2, 0)$ ha valore 2

Esempio (1/3): la feasibility pump sull'esempio dell'auto

- L'arrotondamento della soluzione LP dà $\bar{\boldsymbol{x}}^0 = (4, 3)$, che è inammissibile

k	proiezione $\tilde{\boldsymbol{x}}^k$	$\Delta(\tilde{\boldsymbol{x}}^k, \bar{\boldsymbol{x}}^{k-1})$	punto intero successivo
1	$(\frac{7}{2}, \frac{5}{2})$	1	$[\tilde{\boldsymbol{x}}^1] = \bar{\boldsymbol{x}}^0$: flip di x_1, $\bar{\boldsymbol{x}}^1 = (3, 3)$
2	$(3, \frac{37}{14})$	$\frac{5}{14}$	$[\tilde{\boldsymbol{x}}^2] = \bar{\boldsymbol{x}}^1$: flip di x_2, $\bar{\boldsymbol{x}}^2 = (3, 2)$
3	$(3, 2)$	0	stop: ammissibile

Esempio (2/3): la feasibility pump sull'esempio dell'auto

- All'iterazione 1 le due componenti di $\tilde{\boldsymbol{x}}^1$ sono alla stessa distanza $\frac{1}{2}$ da $\bar{\boldsymbol{x}}^0$, e il flip prende la prima
- La pompa ha bisogno di **due flip** e restituisce $(3, 2)$, di valore 13
- È ammissibile, ma non ottima ($z(\mathrm{ILP}) = 14$), perché dopo il primo LP la funzione obiettivo viene ignorata

Esempio (3/3): la feasibility pump su *P*72

- L'arrotondamento e il diving falliscono (si vedano gli esempi delle sezioni precedenti)
- La pompa arrotonda la soluzione LP a $\bar{\boldsymbol{x}}^0 = (1, 0, 2, 3, 1)$, che viola il primo vincolo
- Ha bisogno di tre proiezioni e di un flip (di x_3, da 2 a 1) per arrivare alla soluzione ammissibile

$$\bar{\boldsymbol{x}} = (0, 0, 1, 3, 1), \qquad f(\bar{\boldsymbol{x}}) = 2,$$

molto lontana da $z(\mathrm{ILP}) = 7$ e dall'upper bound 8

La pompa è un metodo per trovare una *prima* soluzione ammissibile,
e lo fa bene.

La qualità della soluzione è compito dei metodi di miglioramento
delle prossime sezioni, che partono da una soluzione incumbent.

Riepilogo

- La **feasibility pump** alterna un arrotondamento (un punto intero) e una proiezione (il punto più vicino di $\mathcal{L}$, un LP) finché i due punti non coincidono
- La distanza $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ è la distanza L_1 sulle variabili intere: lineare per variabili binarie, con variabili ausiliarie d_j per le intere generali
- I cicli corti si rompono con un **flip**, quelli lunghi con un **restart** casuale
- La pompa ignora la funzione obiettivo: su $P72$ trova una soluzione di valore 2, contro il valore ottimo 7

Migliorare l'incumbent con un sotto-MIP

Introduzione: Migliorare l'incumbent con un sotto-MIP

- I metodi delle sezioni precedenti costruiscono una soluzione a partire dal rilassamento lineare
- I tre metodi di questa sezione partono da una soluzione **incumbent** $\bar{\boldsymbol{x}}$, calcolata con una qualunque euristica — arrotondamento, diving, feasibility pump — e cercano di migliorarla
- Vediamo lo schema comune (un sotto-MIP con un vincolo di cutoff), poi RINS, il local branching e la proximity search

Il sotto-MIP

- Seguono tutti lo stesso schema: aggiungono al MIP alcuni vincoli che lo restringono a un insieme di soluzioni *legate a* $\bar{\boldsymbol{x}}$, e risolvono il problema ristretto, detto **sotto-MIP**, con il solver MIP

- Il sotto-MIP è un intorno di $\bar{\boldsymbol{x}}$, ma molto ampio, esplorato in modo esatto dal solver e non per enumerazione

- È l'idea della **large neighbourhood search** (ricerca in un intorno ampio), con il solver come metodo di riparazione

Il vincolo di cutoff

- Interessano solo le soluzioni *migliori* dell'incumbent
- Con un passo di miglioramento $\theta > 0$, il sotto-MIP contiene il **vincolo di cutoff**

$$f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta \quad \text{(minimizzazione)},$$

$$f(\boldsymbol{x}) \geq f(\bar{\boldsymbol{x}}) + \theta \quad \text{(massimizzazione)}.$$

- Quando f assume valori interi su tutte le soluzioni ammissibili, come in tutti gli esempi di questo capitolo, $\theta = 1$ esclude esattamente le soluzioni che non sono migliori di $\bar{\boldsymbol{x}}$

Ottimo locale, fissaggio rigido e morbido

- Se il sotto-MIP con il cutoff è inammissibile, l'intorno non contiene soluzioni migliori: il solver ha *dimostrato* che $\bar{\boldsymbol{x}}$ è un ottimo locale per quell'intorno

- I metodi differiscono per *quali* vincoli definiscono l'intorno:
  - RINS fissa variabili (**fissaggio rigido**, hard fixing)
  - il local branching limita il numero di variabili che cambiano e la proximity search lo minimizza (**fissaggio morbido**, soft fixing)

Il primo metodo confronta l'incumbent
con la soluzione LP, e fissa le variabili
su cui le due concordano: **RINS**.

Definizione 8: RINS

RINS (Relaxation Induced Neighbourhood Search, ricerca nell'intorno indotto dal rilassamento; Danna, Rothberg e Le Pape, 2005) confronta l'incumbent $\bar{\mathbf{x}}$ con la soluzione $\tilde{\mathbf{x}}$ del rilassamento lineare. Le variabili intere su cui le due soluzioni concordano vengono fissate al loro valore comune; il sotto-MIP sulle variabili rimanenti, con il vincolo di cutoff, viene risolto dal solver MIP.

- Lo pseudocodice di RINS è nelle prossime due slide

Algorithm: RINS (1/2)

Input : MIP; incumbent $\bar{\boldsymbol{x}}$; passo di miglioramento θ
Output : Una soluzione buona almeno quanto $\bar{\boldsymbol{x}}$

```
1 function RINS(MIP, x̄, θ)
2 |   x̃ := soluzione ottima del rilassamento lineare
3 |   for j ∈ 𝒟 do
4 |   |   if x̃_j = x̄_j then
5 |   |   |   fissa x_j := x̄_j   // LP e incumbent concordano
6 |   |   end
7 |   end
```

Algorithm: RINS (2/2)

function *RINS(MIP,* $\bar{\boldsymbol{x}}$*,* θ*)* (continua)

8 aggiungi il vincolo di cutoff $f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta$

9 risolvi il sotto-MIP con il solver MIP (con un limite sui nodi)

10 **if** *il sotto-MIP ha una soluzione ammissibile* $\boldsymbol{x}'$ **then**

11 **return** $\boldsymbol{x}'$ `// migliore di` $\bar{\boldsymbol{x}}$

12 **else**

13 **return** $\bar{\boldsymbol{x}}$ `// nessuna soluzione migliore nell'intorno`

14 **end**

15 **end**

Perché l'accordo

- La soluzione LP è ottimista e forse inammissibile, l'incumbent è ammissibile e forse scadente: due consiglieri con difetti opposti
- Dove concordano, il valore è probabilmente giusto anche nelle buone soluzioni
- Dove non concordano, c'è qualcosa da decidere, e la decisione è lasciata al solver

Ampiezza dell'intorno e fissaggio rigido

- **L'ampiezza dell'intorno non si sceglie.** È il numero di variabili su cui $\tilde{\boldsymbol{x}}$ e $\bar{\boldsymbol{x}}$ non concordano: piccolo se l'incumbent è già vicino alla soluzione LP, grande altrimenti. Per questo il sotto-MIP si risolve con un limite sul numero di nodi

- **RINS dentro il branch-and-bound.** Danna, Rothberg e Le Pape applicano RINS nei nodi dell'albero di branch-and-bound, con la soluzione LP del nodo: nodi più profondi hanno soluzioni LP diverse, quindi intorni diversi dello stesso incumbent. Ogni solver MIP moderno esegue RINS in questo modo

- **Fissaggio rigido.** Una variabile fissata non può cambiare nel sotto-MIP. Se una soluzione migliore richiede di cambiare una variabile su cui $\tilde{\boldsymbol{x}}$ e $\bar{\boldsymbol{x}}$ concordano, RINS non può trovarla

Esempio (1/2): RINS su *P7*

- L'incumbent è $\bar{\boldsymbol{x}} = (2, 0)$, di valore 6 (arrotondamento per difetto), e la soluzione LP è $\tilde{\boldsymbol{x}} = \left(\frac{5}{2}, 0\right)$. Concordano su $x_2 = 0$, che viene fissata. Il sotto-MIP è

$$\begin{aligned} \max \quad & 3\,x_1 \\ \text{soggetto a} \quad & x_1 \leq 12, \\ & 2\,x_1 \leq 5, \\ & 3\,x_1 \geq 7, \\ & x_1 \in \mathbb{Z}_{\geq 0} \end{aligned}$$

- È inammissibile: il secondo vincolo dà $x_1 \leq 2$, il cutoff $x_1 \geq 3$. RINS restituisce l'incumbent
- La soluzione ottima $(2, 1)$ ha $x_2 = 1$, ma x_2 è fissata a 0 perché l'LP e l'incumbent concordano su di essa

Esempio (2/2): RINS su *P*42

- L'incumbent è $\bar{\boldsymbol{x}} = (2, 1, 0)$, di valore 8 (arrotondamento per difetto), e $\tilde{\boldsymbol{x}} = \left(\frac{5}{2}, \frac{3}{2}, 0\right)$
- Concordano solo su $x_3 = 0$
- Il sotto-MIP su x_1, x_2 con il cutoff $3x_1 + 2x_2 \geq 9$ viene risolto da Gurobi, che restituisce $(2, 2, 0)$, di valore 10
- **RINS trova una soluzione ottima**

Su $P7$ la variabile che deve cambiare è proprio quella
su cui l'LP e l'incumbent concordano;

su $P42$ l'accordo riguarda una variabile
che non deve cambiare,
e l'intorno contiene l'ottimo.

Il vincolo di local branching

- RINS decide *quali* variabili possono cambiare. Il local branching (Fischetti e Lodi, 2003) decide solo *quante* possono cambiare, e lascia al solver la scelta di quali
- Per un MIP le cui variabili intere sono binarie (oppure, con le variabili ausiliarie d_j della distanza, intere generali), il **vincolo di local branching** di raggio $k \in \mathbb{Z}_{\geq 1}$ è

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = \sum_{j \in \mathscr{D}:\ \bar{x}_j = 1} (1 - x_j) + \sum_{j \in \mathscr{D}:\ \bar{x}_j = 0} x_j \leq k$$

- $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ è la distanza della feasibility pump, per variabili binarie: sui punti binari conta le variabili il cui valore differisce da quello dell'incumbent
- Il vincolo è lineare, quindi si può aggiungere al MIP come qualunque altro vincolo

Definizione 9: local branching

Il local branching risolve, con il solver MIP, il sotto-MIP formato dal MIP, dal vincolo di cutoff e dal vincolo di local branching $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$. *Se trova una soluzione migliore, la nuova soluzione diventa l'incumbent e l'intorno viene centrato su di essa. Se dimostra che l'intorno non contiene soluzioni migliori, il raggio k viene aumentato. In entrambi i casi l'intorno esplorato viene escluso dai sotto-MIP successivi con il vincolo inverso* $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \geq k + 1$.

Algorithm: Local branching (1/2)

Input : MIP; incumbent $\bar{\boldsymbol{x}}$; raggio k; raggio massimo $k_{\max}$; passo k_{step}; passo di miglioramento θ

Output : Una soluzione buona almeno quanto $\bar{\boldsymbol{x}}$

1 **function** *LocalBranching(MIP, $\bar{\boldsymbol{x}}$, k, $k_{\max}$, k_{step}, θ)*

2 $\quad \mathscr{R} := \emptyset$ `// vincoli inversi aggiunti finora`

3 $\quad$ **while** $k \leq k_{\max}$ **do**

4 $\quad\quad$ risolvi il MIP con i vincoli $\mathscr{R}$, $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$ e $f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta$

5 $\quad\quad$ $\mathscr{R} \leftarrow \mathscr{R} \cup \left\{\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \geq k + 1\right\}$ `// non esplorarlo mai più`

Algorithm: Local branching (2/2)

function *LocalBranching(MIP,* $\bar{\boldsymbol{x}}$*,* k*,* $k_{\max}$*,* k_{step}*,* θ*)* (continua)

    while $k \leq k_{\max}$ (continua) **do**

6   **if** *si trova una soluzione ammissibile* $\boldsymbol{x}'$ **then**

7            $\bar{\boldsymbol{x}} \leftarrow \boldsymbol{x}'$ // ricentra l'intorno

8   **else**

9            $k \leftarrow k + k_{step}$ // nessuna soluzione migliore entro distanza k

10   **end**

11   **end**

12   **return** $\bar{x}$

13 **end**

Fissaggio morbido

- Con $|\mathscr{D}|$ variabili binarie, l'intorno $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$ contiene

$$\sum_{i=0}^{k} \binom{|\mathscr{D}|}{i}$$

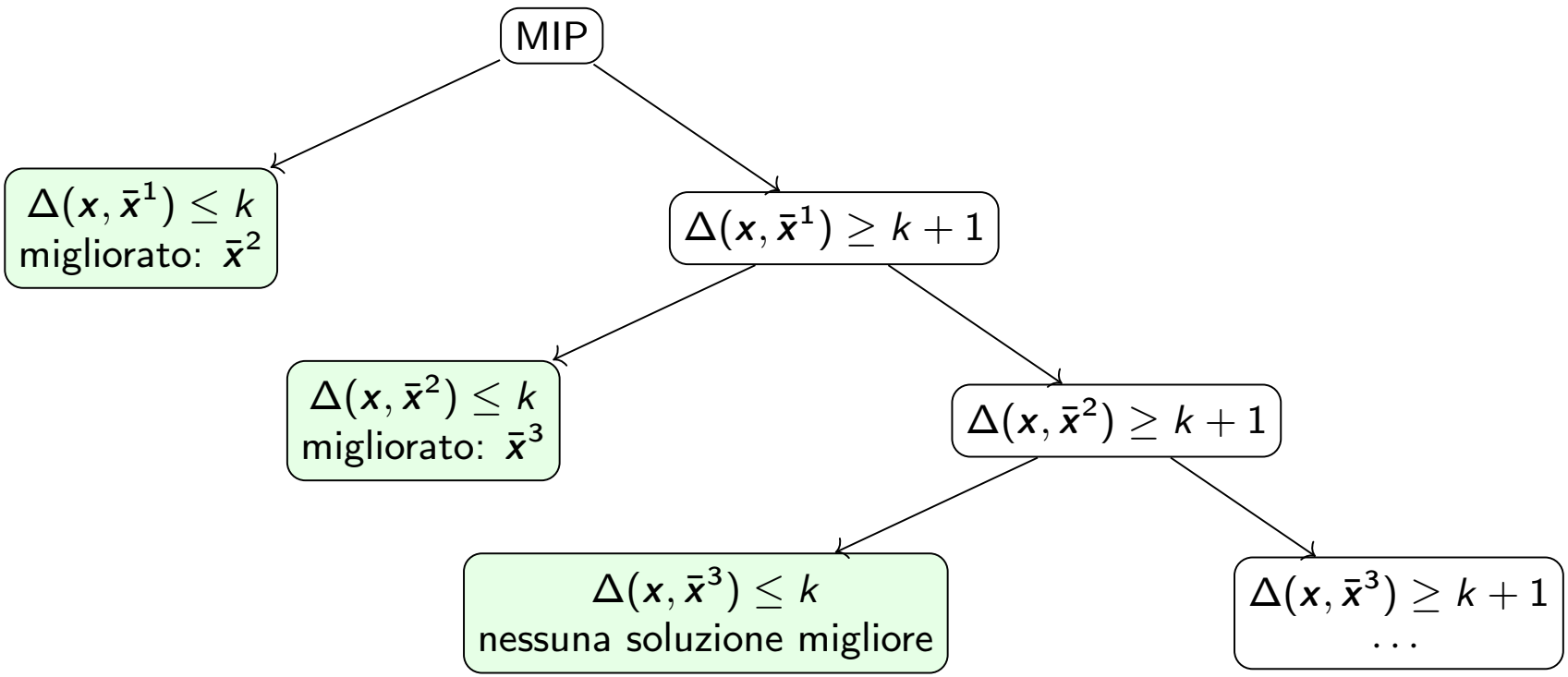
  punti binari
- Per k piccolo è molto più piccolo dell'intero spazio, ma non si vincola a nessuna variabile in particolare

- Possono cambiare k variabili qualsiasi

Uno schema di branching

- Il nome viene dall'uso originale del vincolo come *regola di branching*: si risolve prima il ramo sinistro $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$, e il ramo destro $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \geq k + 1$ contiene il resto del problema
- Se il ramo sinistro migliora l'incumbent, il ramo destro viene suddiviso di nuovo attorno al nuovo incumbent
- L'albero è una lunga catena di rami destri (slide successiva)

L'albero del local branching



Il raggio e la ricerca locale

- **Il raggio.** k deve essere abbastanza piccolo perché il sotto-MIP si risolva in fretta, e abbastanza grande perché l'intorno contenga soluzioni migliori
- Il metodo originale usa valori piccoli rispetto a $|\mathscr{D}|$, un limite di tempo su ogni sotto-MIP, e una *diversificazione* — un k più grande, o un salto verso una soluzione diversa — quando non trova miglioramenti
- **Local branching e ricerca locale.** Con $k = 1$ l'intorno contiene i punti ottenuti da $\bar{\boldsymbol{x}}$ cambiando di un'unità una sola variabile intera (una variabile binaria viene invertita); con k più grande contiene i punti che differiscono da $\bar{\boldsymbol{x}}$ in più variabili, o di più unità
- È una ricerca locale il cui intorno viene esplorato in modo esatto dal solver

Esempio (1/3): local branching su *P7*

- Si parte da $\bar{\boldsymbol{x}}^1 = (2, 0)$, di valore 6, con $k = 1$, $k_{\max} = 3$, $k_{step} = 1$ e $\theta = 1$
- $k = 1$. Il sotto-MIP con $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^1) \leq 1$ e il cutoff $3x_1 + x_2 \geq 7$ restituisce $(2, 1)$, di valore 7: il nuovo incumbent $\bar{\boldsymbol{x}}^2$, trovato a distanza 1 (la variabile x_2 cambia di un'unità). Si aggiunge $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^1) \geq 2$
- $k = 1, 2, 3$, **attorno a** $\bar{\boldsymbol{x}}^2$**.** Nessuna soluzione di valore almeno 8: k diventa $4 > k_{\max}$ e il metodo si ferma con $(2, 1)$, l'ottimo
- Dove RINS falliva, il local branching lascia al solver la scelta di cambiare x_2

Esempio (2/3): local branching su *P*72

- Si parte dalla soluzione della feasibility pump, $\bar{\boldsymbol{x}}^1 = (0, 0, 1, 3, 1)$, di valore 2, con gli stessi parametri

k	attorno a	soluzione restituita	valore
1	$\bar{\boldsymbol{x}}^1$	$\bar{\boldsymbol{x}}^2 = (0, 0, 1, 2, 1)$, a distanza 1	4
1	$\bar{\boldsymbol{x}}^2$	nessuna soluzione di valore ≥ 5	—
2	$\bar{\boldsymbol{x}}^2$	$\bar{\boldsymbol{x}}^3 = (0, 1, 1, 2, 0)$, a distanza 2	7
2, 3	$\bar{\boldsymbol{x}}^3$	nessuna soluzione di valore ≥ 8	—

Esempio (3/3): local branching su *P*72

- Con $k_{\max} = 3$ il metodo si ferma con valore 7, l'ottimo, dopo due miglioramenti
- Il raggio passa da 1 a 2 perché il miglioramento richiede di cambiare insieme x_2 e x_5, cioè due unità di distanza

Il local branching esegue una ricerca locale con un intorno
la cui ampiezza è controllata da k, esplorato in modo esatto dal solver,
e allarga l'intorno quando resta bloccato:
è proprio l'idea della variable neighbourhood search,
con il solver che esegue l'esplorazione.

Il local branching mantiene la funzione obiettivo
e vincola la distanza.

La **proximity search** di Fischetti e Monaci (2014)
fa il contrario: mantiene il cutoff come vincolo
e *minimizza* la distanza.

Definizione 10: proximity search

*Il **sotto-MIP di prossimità** attorno all'incumbent $\bar{\boldsymbol{x}}$ si scrive*

$$
\begin{aligned}
\min \quad & \Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \\
\text{soggetto a} \quad & A\boldsymbol{x} \leq b, \\
& l_j \leq x_j \leq u_j, && \forall j \in \{1, 2, \ldots, N\}, \\
& f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta, \\
& x_j \in \mathbb{Z}, && \forall j \in \mathscr{D}.
\end{aligned}
$$

La proximity search lo risolve, sostituisce l'incumbent con la sua soluzione e ripete, finché il sotto-MIP di prossimità non diventa inammissibile.

Algorithm: Proximity search (1/2)

Input : MIP; incumbent $\bar{\boldsymbol{x}}$; passo di miglioramento θ
Output : Una soluzione buona almeno quanto $\bar{\boldsymbol{x}}$

```
1 function ProximitySearch(MIP, x̄, θ)
2     while vero do
3         risolvi min{Δ(x, x̄) :
            x ammissibile per il MIP, f(x) ≤ f(x̄) − θ}
4         if il sotto-MIP di prossimità è inammissibile then
5             return x̄ // nessuna soluzione migliore di
                f(x̄) − θ
6         end
```

Algorithm: Proximity search (2/2)

function *ProximitySearch(MIP, $\bar{\boldsymbol{x}}$, θ)* (continua)
  while *vero* (continua) **do**
7 $\boldsymbol{x}' :=$ la sua soluzione ottima
8 fissa $x_j := x'_j$ per $j \in \mathscr{D}$ e riottimizza f sulle variabili
   continue
9 $\bar{\boldsymbol{x}} \leftarrow \boldsymbol{x}'$ `// la soluzione migliorante più vicina`
  end
10 **end**
11

Perché minimizzare la distanza

- Spesso è proprio la funzione obiettivo originale a rendere difficile un MIP: il suo rilassamento lineare spinge il solver verso punti frazionari lontani dalle soluzioni ammissibili
- Nel sotto-MIP di prossimità la funzione obiettivo è la distanza, e la soluzione LP resta vicina a $\bar{\boldsymbol{x}}$, che è quasi ammissibile: le euristiche del solver funzionano bene, e di solito si trova in fretta una soluzione ammissibile
- In pratica il sotto-MIP di prossimità non si risolve all'ottimo: qualunque soluzione ammissibile è un miglioramento

Nessun raggio, la parte continua, il passo θ

- **Nessun raggio da scegliere.** L'ampiezza della mossa la decide il sotto-MIP stesso: il più piccolo cambiamento che migliora la funzione obiettivo di almeno θ

- **Riottimizzare la parte continua.** La soluzione del sotto-MIP ha la parte intera giusta, ma le sue variabili continue sono state scelte per minimizzare la distanza, non f. Fissando le variabili intere e risolvendo l'LP sulle variabili continue si ottiene il vero valore del nuovo incumbent

- **Il passo θ.** Un θ grande chiede miglioramenti grandi, quindi mosse ampie e sotto-MIP difficili; un θ piccolo dà sotto-MIP facili e molte iterazioni. Con $\theta = 1$ e valori interi della funzione obiettivo, il metodo dimostra anche l'ottimalità

Proposizione 1

Se f assume valori interi su tutte le soluzioni ammissibili del MIP e il sotto-MIP di prossimità attorno a $\bar{\boldsymbol{x}}$ con $\theta = 1$ è inammissibile, allora $f(\bar{\boldsymbol{x}})$ è il valore ottimo del MIP.

Dimostrazione

- La regione ammissibile del sotto-MIP di prossimità è l'insieme delle soluzioni ammissibili $\boldsymbol{x}$ del MIP con

$$f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - 1;$$

  non dipende dalla funzione obiettivo Δ
- Se è vuota, ogni soluzione ammissibile ha

$$f(\boldsymbol{x}) > f(\bar{\boldsymbol{x}}) - 1, \quad \text{quindi} \quad f(\boldsymbol{x}) \geq f(\bar{\boldsymbol{x}}),$$

  perché entrambi i valori sono interi
- Dunque $\bar{\boldsymbol{x}}$ è ottima



Esempio (1/2): proximity search su $P7$

- Da $\bar{\boldsymbol{x}} = (2, 0)$, di valore 6: la soluzione più vicina di valore almeno 7 è $(2, 1)$, a distanza 1
- Attorno a essa, nessuna soluzione ha valore almeno 8 (il bound LP è $\frac{15}{2}$)
- Per la proposizione precedente, $(2, 1)$ è **ottima**

Esempio (2/2): proximity search su *P*72

- Dalla soluzione della feasibility pump, di valore 2:

iterazione	cutoff	min Δ	soluzione restituita	valore
1	$f(\boldsymbol{x}) \geq 3$	1	$(0, 0, 1, 2, 1)$	4
2	$f(\boldsymbol{x}) \geq 5$	2	$(0, 1, 1, 2, 0)$	7
3	$f(\boldsymbol{x}) \geq 8$	—	inammissibile	—

- Alla terza iterazione il sotto-MIP è inammissibile, e la proposizione dimostra che 7 è ottimo, mentre l'upper bound $\lfloor z(\mathrm{LP}) \rfloor = 8$ della radice non basta a dimostrarlo

La dimostrazione di ottimalità viene dal solver,
che dimostra l'inammissibilità dell'ultimo sotto-MIP
con i propri bound duali:

la proximity search, usata con $\theta = 1$,
è un metodo esatto che migliora l'incumbent
strada facendo.

Riepilogo (1/2)

Lo schema comune:

- Un **sotto-MIP**: il MIP ristretto a soluzioni legate all'incumbent, esplorato in modo esatto dal solver (large neighbourhood search)
- Il **vincolo di cutoff** tiene solo le soluzioni migliori; un sotto-MIP inammissibile dimostra un ottimo locale

RINS:

- Fissa le variabili su cui la soluzione LP e l'incumbent concordano (**fissaggio rigido**)
- Trova l'ottimo su $P42$, fallisce su $P7$, dove la variabile da cambiare è una di quelle fissate

Riepilogo (2/2)

Local branching:

- Aggiunge $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$ (**fissaggio morbido**), si ricentra quando migliora, allarga k quando resta bloccato
- Raggiunge l'ottimo su $P7$ e su $P72$

Proximity search:

- Minimizza $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ soggetto al cutoff: nessun raggio da scegliere
- Con $\theta = 1$ e valori interi della funzione obiettivo, un sotto-MIP di prossimità inammissibile **dimostra l'ottimalità** ($P7$, $P72$)

Relax-and-fix e fix-and-optimize

Introduzione: Relax-and-fix e fix-and-optimize

- Alcuni MIP sono troppo grandi per essere risolti in una volta, ma le loro variabili si presentano in blocchi
- Il **relax-and-fix** costruisce una soluzione blocco per blocco; il **fix-and-optimize** la migliora finestra per finestra
- Eseguiamo entrambi su $P46$, e il fix-and-optimize anche su $P72$

Blocchi di variabili

- Le variabili di molti MIP si presentano in **blocchi** con un ordine naturale, per esempio perché sono indicizzate da una sequenza, e le decisioni di un blocco interagiscono soprattutto con quelle dei blocchi vicini
- I due metodi di questa sezione sfruttano una partizione delle variabili intere in blocchi: risolvono una sequenza di MIP piccoli invece di uno grande
- Una volta fissate le variabili intere, il resto è un LP: le decisioni difficili sono le variabili intere, e il rilassamento lineare è lo strumento che stima l'effetto delle decisioni non ancora prese

Definizione 11: relax-and-fix

Il relax-and-fix suddivide le variabili intere in blocchi $B_1, B_2, \ldots, B_p$*, in un ordine dato. Al passo r risolve il MIP in cui le variabili di* B_r *sono intere, quelle di* $B_1, \ldots, B_{r-1}$ *sono fissate ai valori decisi nei passi precedenti, e quelle di* $B_{r+1}, \ldots, B_p$ *sono rilassate (continue). Poi fissa le variabili di* B_r *ai loro valori e passa al blocco successivo. La soluzione dell'ultimo passo è ammissibile.*

Algorithm: Relax-and-fix

Input : MIP; partizione $B_1, B_2, \ldots, B_p$ di $\mathscr{D}$, in un ordine dato

Output : Soluzione ammissibile, oppure FALLIMENTO

```
1  function RelaxAndFix(MIP, B_1, ..., B_p)
2      for r := 1 to p do
3          risolvi il MIP con: x_j fissata per j ∈ B_1 ∪ ··· ∪ B_{r-1}, x_j
             intera per j ∈ B_r, x_j continua per j ∈ B_{r+1} ∪ ··· ∪ B_p
4          if è inammissibile then
5              return FALLIMENTO
6          end
7          x' := la sua soluzione ottima
8          fissa x_j := x'_j per ogni j ∈ B_r
9      end
10     return x'        // tutte le variabili intere sono
         intere
11 end
```

Perché funziona, e la sua miopia

- **Perché.** Le decisioni dei primi blocchi si prendono con cura (intere), mentre le altre sono solo stimate (rilassate). Ogni MIP ha le variabili intere di un solo blocco, quindi è piccolo
- **Miopia.** I blocchi rilassati sono ottimisti: il rilassamento può comprare una frazione di decisione per una frazione del suo costo. Una decisione che sembra buona perché il resto costa poco può rivelarsi cattiva quando il resto diventa intero. Le decisioni fissate possono perfino rendere inammissibile un passo successivo
- **Varianti.** Blocchi più grandi danno soluzioni migliori e MIP più difficili; con blocchi *sovrapposti* si risolve $B_r \cup B_{r+1}$ come intero ma si fissa solo B_r, per vedere un po' più lontano

Esempio (1/3): relax-and-fix su $P46$

- Le variabili intere sono x_1, x_2, x_3
- Il rilassamento lineare ha valore $\frac{499}{25}$, l'upper bound è 19 e $z(\mathrm{ILP}) = 16$, raggiunto da $(0, 1, 1)$
- **Una variabile per blocco,** $B_1 = \{x_1\}$, $B_2 = \{x_2\}$, $B_3 = \{x_3\}$:

passo r	valore del MIP del passo r	soluzione del MIP	fissaggio
1	$\frac{296}{15} \approx 19.73$	$\left(0, \frac{26}{15}, \frac{14}{15}\right)$	$x_1 := 0$
2	$\frac{56}{3} \approx 18.67$	$\left(0, 2, \frac{2}{3}\right)$	$x_2 := 2$
3	12	$(0, 2, 0)$	$x_3 := 0$

Esempio (2/3): relax-and-fix su *P*46

- Il risultato è $(0, 2, 0)$, di valore 12, con un gap di

$$\frac{19 - 12}{12} \approx 58.3\%$$

- La decisione sbagliata si prende al passo 2: con x_3 rilassata, $x_2 = 2$ dà $\frac{56}{3} \approx 18.67$ e $x_2 = 1$ dà $\frac{122}{7} \approx 17.43$, quindi $x_2 := 2$

- Ma allora il secondo vincolo lascia spazio solo per $x_3 \leq \frac{2}{3}$, e x_3 diventa 0 quando è intera

Esempio (3/3): relax-and-fix su $P46$

- **Blocchi più grandi,** $B_1 = \{x_1\}$**,** $B_2 = \{x_2, x_3\}$**.** Il passo 1 è lo stesso
- Al passo 2 il MIP su x_2, x_3 si risolve con entrambe le variabili intere: restituisce $(0, 1, 1)$, di valore 16, l'**ottimo**
- L'ampiezza dei blocchi bilancia la qualità della soluzione e la difficoltà dei MIP: il blocco più grande vede l'interazione fra x_2 e x_3 che i blocchi di una sola variabile non colgono

Definizione 12: fix-and-optimize

Il fix-and-optimize migliora un incumbent $\bar{\mathbf{x}}$. Considera una sequenza di finestre $W_1, W_2, \ldots, W_q \subseteq \mathscr{D}$, tipicamente periodi consecutivi. Per ogni finestra fissa tutte le variabili intere fuori dalla finestra ai loro valori in $\bar{\mathbf{x}}$, e risolve il sotto-MIP sulle variabili della finestra, con il vincolo di cutoff. Una soluzione migliore diventa l'incumbent. Le passate sulle finestre si ripetono finché un'intera passata non porta miglioramenti.

Algorithm: Fix-and-optimize

Input : MIP; incumbent $\bar{\boldsymbol{x}}$; finestre $W_1, \ldots, W_q$; passo di miglioramento θ

Output : Una soluzione buona almeno quanto $\bar{\boldsymbol{x}}$

```
1  function FixAndOptimize(MIP, x̄, W_1, ..., W_q, θ)
2      improved := vero
3      while improved do
4          improved ← falso
5          for r := 1 to q do
6              risolvi il MIP con x_j := x̄_j per j ∈ 𝒟 \ W_r e
                 f(x) ≤ f(x̄) − θ
7              if si trova una soluzione ammissibile x' then
8                  x̄ ← x';    improved ← vero
9              end
10         end
11     end
12     return x̄
13 end
```

Esempio (1/3): fix-and-optimize su $P46$

- Si parte dalla soluzione del relax-and-fix, $\bar{\boldsymbol{x}} = (0, 2, 0)$, di valore 12, con le finestre $W_1 = \{x_1\}$ e $W_2 = \{x_2, x_3\}$ e $\theta = 1$

finestra	incumbent dopo la finestra	
$\{x_1\}$	12	nessun miglioramento ($x_1 = 1$ viola il primo vincolo)
$\{x_2, x_3\}$	16	$\bar{\boldsymbol{x}} = (0, 1, 1)$
seconda passata, entrambe le finestre	16	nessun miglioramento: stop

Esempio (2/3): fix-and-optimize su $P46$

- Con le tre finestre di una variabile ciascuna, $\{x_1\}$, $\{x_2\}$, $\{x_3\}$, nessuna finestra migliora $(0, 2, 0)$: è un ottimo locale per quelle finestre
- La finestra $\{x_2, x_3\}$ rivede la decisione sbagliata del relax-and-fix e raggiunge l'ottimo 16

Esempio (3/3): fix-and-optimize su *P*72

- Si parte dalla soluzione della feasibility pump, $(0, 0, 1, 3, 1)$, di valore 2, con le finestre $\{x_1, x_2\}$, $\{x_3, x_4\}$, $\{x_5\}$
- La prima finestra non porta miglioramenti; la seconda dà $(0, 0, 2, 4, 1)$, di valore 5; la terza e la seconda passata non ne portano
- Il metodo si ferma con $5 < 7 = z(\mathrm{ILP})$: un ottimo locale rispetto a queste finestre
- La soluzione ottima $(0, 1, 1, 2, 0)$ ne differisce in x_2, x_3, x_4 e x_5, più variabili di quante ne contenga una qualunque finestra

Ricerca locale a finestre

- Il fix-and-optimize è una **ricerca locale** le cui mosse cambiano tutte insieme le variabili intere di una finestra, esplorate in modo esatto dal solver: l'equivalente, nella programmazione intera, degli intorni della ricerca locale
- Si ferma in un ottimo locale rispetto a tutte le finestre
- Relax-and-fix e fix-and-optimize sono complementari: il primo costruisce una soluzione blocco per blocco, il secondo ripara le decisioni miopi guardando più blocchi insieme

Riepilogo

- **Relax-and-fix**: un blocco intero, i precedenti fissati, i successivi rilassati; poi si fissa il blocco e si passa al successivo
- È **miope**: su $P46$ i blocchi di una variabile danno 12, il blocco $\{x_2, x_3\}$ dà l'ottimo 16
- **Fix-and-optimize**: si riottimizzano le variabili di una finestra, il resto fissato all'incumbent, con il cutoff; si ripetono le passate finché non ci sono miglioramenti
- Si ferma in un ottimo locale per le finestre: 16 su $P46$, $5 < 7$ su $P72$

Le euristiche dentro il solver

Introduzione: Le euristiche dentro il solver

- Un solver MIP moderno non è solo un algoritmo branch-and-bound: esegue, in automatico, gran parte delle euristiche di questo capitolo
- Vediamo quando Gurobi le esegue, e i parametri che controllano lo sforzo dedicato a esse
- Vediamo anche come passare al solver una soluzione calcolata fuori da esso

Quando vengono eseguite le euristiche

- Il capitolo «Solver MIP: che cosa fanno» mostra log reali. In Gurobi:
  - *prima dell'LP della radice*, euristiche che non hanno bisogno del rilassamento cercano una prima soluzione
  - *alla radice*, dopo l'LP, vengono eseguiti l'arrotondamento, diverse euristiche di diving e la feasibility pump
  - *durante il branch-and-bound*, RINS e altre euristiche basate su sotto-MIP vengono eseguite in alcuni nodi, con la soluzione LP del nodo
- I nuovi incumbent sono segnalati nel log da una riga che inizia con `H` (euristica) invece che con $*$ (soluzione LP intera di un nodo)

I parametri delle euristiche

- Lo sforzo dedicato alle euristiche è controllato da parametri. I più importanti sono:

parametro	default	significato
`Heuristics`	0.05	frazione del lavoro dedicata alle euristiche (da 0 a 1)
`MIPFocus`	0	1: puntare a trovare buone soluzioni ammissibili più che a migliorare il bound
`RINS`	-1	frequenza di RINS (ogni n nodi; -1 automatica, 0 disattivata)
`PumpPasses`	-1	passate della feasibility pump alla radice (-1 automatico)
`SubMIPNodes`	500	limite sui nodi dei sotto-MIP di RINS e di euristiche simili
`NoRelHeurWork`	0	lavoro dedicato all'euristica senza rilassamento prima della radice
`ImproveStartGap`	0	quando il gap raggiunge questo valore, si passa a migliorare solo l'incumbent

MIP start

- Una soluzione euristica calcolata fuori dal solver — con uno qualunque dei metodi di questo capitolo — gli si può passare come **MIP start** (l'attributo `Start` delle variabili)
- Il solver parte da essa come incumbent, la usa per la potatura e per RINS, e non restituisce mai niente di peggio
- I numeri dipendono dall'istanza, dalla versione del solver e dalla macchina; una singola istanza non dimostra nulla
- La lezione è qualitativa: quando il bound è forte, il tempo di risoluzione è il tempo necessario a trovare una buona soluzione, e a deciderlo sono le euristiche del solver — o un buon MIP start

Riepilogo

- Gurobi esegue euristiche prima dell'LP della radice, alla radice (arrotondamento, diving, feasibility pump) e durante il branch-and-bound (RINS e altre euristiche basate su sotto-MIP); righe `H` nel log
- Parametri come `Heuristics`, `MIPFocus`, `RINS`, `PumpPasses`, `SubMIPNodes` controllano lo sforzo
- Una soluzione euristica esterna entra come **MIP start**

Che cosa danno le euristiche basate sulla PLI

Introduzione: Che cosa danno le euristiche basate sulla PLI

- Una soluzione euristica da sola non dice nulla sulla propria qualità
- Un'euristica basata sulla PLI ha a disposizione un bound duale: calcoliamo il gap di tutte le soluzioni trovate nel capitolo
- Chiudiamo con una tabella che confronta tutti i metodi

Un'euristica che sa quanto è lontana

- Un'euristica fornisce solo un lato del gap, e senza un bound duale non si può dire nulla sulla qualità della sua soluzione

- Un'euristica basata sulla PLI lavora con un modello MIP, e il modello porta con sé il suo rilassamento lineare; eseguita dentro un solver MIP, ha a disposizione anche il bound duale del branch-and-bound

- L'euristica **sa quanto è lontana dall'ottimo** mentre è in esecuzione: il gap di ottimalità

$$\text{gap di ottimalità} = \frac{\text{UB} - \text{LB}}{|\text{LB}|}$$

si calcola in ogni momento, e quando vale 0 la soluzione è *dimostrata* ottima

I gap degli esempi

istanza	soluzione	LB	UB $= \lfloor z(\mathrm{LP}) \rfloor$	gap	certificata da
auto, arrotondamento per difetto	$(3, 2)$	13	15	15.4%	—
auto, diving	$(4, 1)$	14	15	7.1%	proximity search da $(3, 2)$
*P*7, arrotondamento per difetto	$(2, 0)$	6	7	16.7%	—
*P*7, local branching	$(2, 1)$	7	7	0%	il bound
*P*42, RINS	$(2, 2, 0)$	10	10	0%	il bound
*P*46, relax-and-fix	$(0, 2, 0)$	12	19	58.3%	—
*P*46, fix-and-optimize	$(0, 1, 1)$	16	19	18.8%	proximity search
*P*72, feasibility pump	$(0, 0, 1, 3, 1)$	2	8	300%	—
*P*72, local branching	$(0, 1, 1, 2, 0)$	7	8	14.3%	proximity search

Trovare o dimostrare

- Su $P7$ e $P42$ l'arrotondamento per difetto del valore LP è già il valore ottimo: la difficoltà è trovare una soluzione che raggiunga il bound, non dimostrarlo
- Sulle altre istanze il bound $\lfloor z(\mathrm{LP}) \rfloor$ non è stretto, e il certificato viene dalla proximity search (la proposizione di ottimalità) o dal branch-and-bound del solver

Un'euristica basata sulla PLI non si limita a trovare soluzioni:
dentro un solver MIP lavora con un bound duale,
quindi il gap della sua soluzione è sempre noto,
e un gap uguale a 0 certifica l'ottimalità.

I metodi di questo capitolo (1/2)

metodo	richiede l'LP	richiede l'incumbent	fissaggio	idea
arrotondamento	sì	no	—	arrotondare la soluzione LP
diving	sì	guidato: sì	una alla volta	fissare una variabile, risolvere di nuovo l'LP, ripetere
feasibility pump	sì	no	—	alternare arrotondamento e proiezione finché si incontrano
RINS	sì	sì	rigido	fissare dove LP e incumbent concordano, risolvere il sotto-MIP

I metodi di questo capitolo (2/2)

metodo	richiede l'LP	richiede l'incumbent	fissaggio	idea
local branching	no	sì	morbido	sotto-MIP con $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$
proximity search	no	sì	morbido	minimizzare $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ soggetto al cutoff
relax-and-fix	sì	no	a blocchi	intere ora, rilassate dopo, poi fissate
fix-and-optimize	no	sì	a finestre	riottimizzare una finestra, il resto fissato

Riepilogo

- Con un bound duale, il **gap di ottimalità** $\frac{\mathrm{UB}-\mathrm{LB}}{|\mathrm{LB}|}$ di una soluzione euristica è noto in ogni momento
- Sugli esempi i gap vanno dal 300% (feasibility pump su $P72$) allo 0% (local branching su $P7$, RINS su $P42$)
- Quando il bound non è stretto, il certificato viene dalla proximity search o dal branch-and-bound
- I metodi differiscono per ciò che richiedono (LP, incumbent) e per come fissano le variabili (nessun fissaggio, una alla volta, rigido, morbido, a blocchi, a finestre)

Implementazione in Python

Introduzione: Implementazione in `Python`

- Tutti i metodi del capitolo sono implementati in Python con `gurobipy`
- Vediamo come sono organizzati i codici, e due funzioni brevi: l'arrotondamento e la proximity search
- I codici riproducono tutti gli esempi del capitolo

L'organizzazione dei codici

- I codici usano Python 3 e `gurobipy`. Alle variabili di un modello si accede tramite la loro posizione j in `m.getVars()`, e la lista `I` delle posizioni delle variabili intere è l'insieme $\mathscr{D}$; gli indici partono da 0
- Il modulo `common.py` contiene l'ambiente del solver e le operazioni usate da tutti i metodi: il rilassamento lineare, l'arrotondamento $[\cdot]$, il completamento di un punto, la distanza Δ e il vincolo di cutoff
- Il modello degli esempi (`models.py`): `max c x` soggetto a `A x <= b`, con variabili intere in $\mathbb{Z}_{\geq 0}$ e limiti superiori facoltativi

L'arrotondamento in Python

```python
from common import *

def rounding(m, trace=None):
    I = int_index(m)
    x_star = solve_lp(m)
    note(trace, "LP", x_star)
    x_tilde = round_point(x_star, I)
    note(trace, "rounded", x_tilde)
    return evaluate(m, x_tilde)
```

- `solve_lp`: la soluzione ottima del rilassamento lineare
- `round_point`: $\bar{x}_j = [\tilde{x}_j]$ per ogni variabile intera
- `evaluate` è il completamento: `(valore, x)` se ammissibile, `None` altrimenti

La proximity search in Python

```
def proximity_search(m, x_tilde, theta=1, trace=None):
    I = int_index(m)
    z_tilde = obj_of(m, x_tilde)
    while True:
        p = m.copy()
        add_cutoff(p, z_tilde, theta)
        p.setObjective(distance(p, x_tilde, I),
                       GRB.MINIMIZE)
        res = solve_mip(p)
        if res is None:
            note(trace, "infeasible")
            return z_tilde, x_tilde
        z_new, x_new = evaluate(m, res[1])
        note(trace, "found", res[0], z_new, x_new)
        z_tilde, x_tilde = z_new, x_new
```

I file dei codici

- `common.py`, `models.py`: le operazioni comuni e il modello degli esempi
- `rounding.py`, `diving.py`, `feasibility_pump.py`
- `rins.py`, `local_branching.py`, `proximity_search.py`
- `relax_and_fix.py`, `fix_and_optimize.py`
- Tutte le funzioni sono nella cartella `MIP_HEUR_CODE`, insieme a `instances.py`, che contiene le istanze del capitolo e degli esercizi
- Il comando `python3 run_tests.py` esegue tutti gli esempi del capitolo e tutte le istanze degli esercizi, e confronta ogni valore con il valore atteso, scritto nel file

Riepilogo

- Una funzione Python per ogni metodo, costruita sulle operazioni comuni di `common.py`: rilassamento lineare, arrotondamento, completamento, distanza, cutoff
- I sotto-MIP sono copie del modello con limiti fissati o vincoli aggiuntivi, risolte da Gurobi
- `run_tests.py` controlla tutti gli esempi del capitolo e tutte le istanze degli esercizi

Errori tipici

Introduzione: Errori tipici

- Elenchiamo gli errori che si fanno più spesso con le euristiche basate sulla PLI
- Riguardano ciò che ogni metodo può e non può fare: l'arrotondamento, il diving, la pompa, il fissaggio rigido, il cutoff, la dimostrazione di ottimalità e la miopia del relax-and-fix
- Conoscerli aiuta a scegliere il metodo giusto e a leggere correttamente i risultati

Errori tipici

Osservazione 2: Errori tipici (1/3)

- ***Arrotondare un problema con vincoli di uguaglianza.*** *Arrotondamenti indipendenti delle variabili di un vincolo di uguaglianza non lo soddisfano quasi mai: l'arrotondamento fallisce, e serve un diving.*
- ***Credere che un diving possa annullare una decisione.*** *Un diving è un unico cammino dell'albero, senza backtracking: una fissazione sbagliata non viene mai rivista, e una fissazione che rende l'LP inammissibile chiude il diving.*
- ***Usare la feasibility pump per ottenere una buona soluzione.*** *Dopo il primo LP la pompa ignora la funzione obiettivo: su P72 restituisce una soluzione di valore 2, contro il valore ottimo 7.*

Errori tipici

Osservazione: Errori tipici (2/3)

- ***Fidarsi del fissaggio rigido.*** *RINS fissa le variabili su cui l'LP e l'incumbent concordano, quindi non può trovare una soluzione migliore che ne cambi una (P7).*
- ***Dimenticare il vincolo di cutoff.*** *Senza $f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta$ il sotto-MIP restituisce di nuovo la soluzione incumbent; con il vincolo, un sotto-MIP inammissibile dimostra un ottimo locale.*
- ***Dichiarare l'ottimalità senza le ipotesi.*** *La proposizione di ottimalità della proximity search richiede $\theta = 1$ e valori interi della funzione obiettivo su tutte le soluzioni ammissibili.*

Errori tipici

Osservazione: Errori tipici (3/3)

- ***Ignorare la miopia del relax-and-fix.*** *I blocchi rilassati sono ottimisti: una decisione che sembra economica può costringere più avanti a una decisione cattiva (P46).*
- ***Generalizzare da una sola istanza o da una sola esecuzione del solver.*** *Cammini e tempi dipendono dalla versione del solver e dalle regole di spareggio, quindi le lezioni sono qualitative.*

Riepilogo

Errori tipici:

- L'arrotondamento fallisce con i vincoli di uguaglianza; un diving non rivede mai una decisione
- La feasibility pump trova l'ammissibilità, non la qualità
- Il fissaggio rigido può escludere le soluzioni migliori; il vincolo di cutoff è essenziale
- L'ottimalità dalla proximity search richiede $\theta = 1$ e valori interi della funzione obiettivo
- Il relax-and-fix è miope; una sola istanza o una sola esecuzione non dimostra nulla

In sintesi

Introduzione: In sintesi

- Elenchiamo i riferimenti bibliografici del capitolo
- Raccogliamo le idee chiave del capitolo
- Chiudiamo con il cruscotto dell'esempio dell'auto: i bound trovati finora

Riferimenti bibliografici

- T. Achterberg, T. Berthold, Improving the feasibility pump, *Discrete Optimization* 4 (2007) 77–86
- L. Bertacco, M. Fischetti, A. Lodi, A feasibility pump heuristic for general mixed-integer problems, *Discrete Optimization* 4 (2007) 63–76
- E. Danna, E. Rothberg, C. Le Pape, Exploring relaxation induced neighborhoods to improve MIP solutions, *Mathematical Programming* 102 (2005) 71–90
- M. Fischetti, F. Glover, A. Lodi, The feasibility pump, *Mathematical Programming* 104 (2005) 91–104
- M. Fischetti, A. Lodi, Local branching, *Mathematical Programming* 98 (2003) 23–47
- M. Fischetti, M. Monaci, Proximity search for 0-1 mixed-integer convex programming, *Journal of Heuristics* 20 (2014) 709–731

Riepilogo (1/2)

- Le euristiche basate sulla PLI usano il rilassamento lineare per decidere dove cercare e un solver MIP per esplorare in modo esatto un intorno ampio
- L'arrotondamento e il diving costruiscono una soluzione a partire dall'LP; la feasibility pump cerca solo l'ammissibilità; RINS, il local branching e la proximity search migliorano una soluzione incumbent con un sotto-MIP
- Il relax-and-fix costruisce una soluzione blocco per blocco, il fix-and-optimize la migliora finestra per finestra
- Dentro un solver MIP è disponibile il bound duale: il gap della soluzione euristica è noto, e un gap uguale a 0 certifica l'ottimalità

Riepilogo (2/2)

Cruscotto dell'esempio dell'auto. I lower bound e gli upper bound su $z(\mathrm{ILP})$ trovati finora in questo modulo:

Capitolo	Lower bound	Upper bound	Origine
Programmazione lineare intera	12, 14	–	soluzioni euristiche $(4, 0)$ e $(4, 1)$
Rilassamento lineare e bound duali	–	15	$\lfloor z(\mathrm{LP}) \rfloor$, con $z(\mathrm{LP}) = \frac{31}{2}$
Algoritmo dei piani di taglio	14	14	5 tagli frazionari di Gomory
Branch-and-bound basato su LP	14	14	6 problemi LP risolti
Questo capitolo, arrotondamento per difetto	13	15	$(3, 2)$
Questo capitolo, diving	14	15	$(4, 1)$, certificata ottima dalla proximity search